main_freertos udpEcho udpEchoHooks

this is a sdk example of msp432e401y for udp change and give seperate projects for board to board communication with static ip and what changes have to be made syscfg file

To establish board-to-board UDP communication between two MSP432E401Y LaunchPads, you need two distinct project roles:

1. **Board 1 (Transmitter/Client):** Periodically sends a UDP packet to Board 2.
2. **Board 2 (Receiver/Echo Server):** Listens on the static IP/port and processes or replies to packets.

`main_freertos.c` remains identical on both boards. The static IP configuration is handled in SysConfig, while `udpEcho.c` and `udpEchoHooks.c` are updated. main_freertos.c

1. SysConfig (`.syscfg`) Changes for Static IP

Open the project's `.syscfg` file in CCS GUI (or edit in text mode) for each board:

1. In the left panel, navigate to **TI NDK** → **General**.
2. Select **Network IP Configuration**:

   Change configuration mode from **DHCP** to **Static IP**.
3. Set the respective network addresses:

Setting	Board 1 (Client)	Board 2 (Server/Listener)
IP Address	`192.168.1.101`	`192.168.1.102`

Setting	Board 1 (Client)	Board 2 (Server/Listener)
Subnet Mask	`255.255.255.0`	`255.255.255.0`
Gateway IP	`192.168.1.1`	`192.168.1.1`

> Note: If connecting the two LaunchPads directly with a standard Ethernet cable (without a switch or router), the built-in Ethernet PHY on the MSP432E4 auto-negotiates MDI/MDI-X.

2. Board 1 (Transmitter / Client Project)

`udpSender.c` (Replaces `udpEcho.c` on Board 1)

This task binds a local socket, builds a destination packet addressed to Board 2 (`192.168.1.102:1000`), and sends data every 1 second.

c

```c
#include <string.h>
#include <stdint.h>
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

/* BSD socket support */
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/select.h>

#include <ti/display/Display.h>

#define TARGET_IP     "192.168.1.102"
#define TARGET_PORT   1000
#define PACKET_BUF    128

extern Display_Handle display;
extern void fdOpenSession();
```

```
extern void fdCloseSession();
extern void *TaskSelf();

void *udpSenderFxn(void *arg0)
{
    int sock = -1;
    struct sockaddr_in destAddr;
    char buffer[PACKET_BUF];
    int count = 0;
    int bytesSent;

    fdOpenSession(TaskSelf());

    Display_printf(display, 0, 0, "UDP Client Task Started.\n");

    sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
        Display_printf(display, 0, 0, "Error: Socket creation failed.\n");
        goto shutdown;
    }

    memset(&destAddr, 0, sizeof(destAddr));
    destAddr.sin_family      = AF_INET;
    destAddr.sin_port        = htons(TARGET_PORT);
    destAddr.sin_addr.s_addr = inet_addr(TARGET_IP);

    while (1) {
        snprintf(buffer, sizeof(buffer), "Hello from Board 1 [Seq: %d]", count++)

        bytesSent = sendto(sock, buffer, strlen(buffer), 0,
                           (struct sockaddr *)&destAddr, sizeof(destAddr));

        if (bytesSent < 0) {
            Display_printf(display, 0, 0, "Send error\n");
        } else {
            Display_printf(display, 0, 0, "Sent (%d bytes): %s\n", bytesSent, buf
        }

        sleep(1); /* Send every 1 second */
    }

shutdown:
    if (sock >= 0) {
        close(sock);
    }
    fdCloseSession(TaskSelf());
```

```
    return (NULL);
}
```

`udpSenderHooks.c` (Updated for Board 1)

Change the thread initialization in `netIPAddrHook` to trigger `udpSenderFxn`:

udpEchoHooks.c

```c
extern void *udpSenderFxn(void *arg0);

/* Inside netIPAddrHook() ... */
retc = pthread_create(&thread, &attrs, udpSenderFxn, NULL);
if (retc != 0) {
    Display_printf(display, 0, 0, "pthread_create failed\n");
    while (1);
}
```

3. Board 2 (Receiver / Server Project)

`udpReceiver.c` (Modified `udpEcho.c` for Board 2)

Board 2 listens on port `1000`, receives packets sent by Board 1, and prints incoming data to UART: udpEcho.c

```c
#include <string.h>
#include <stdint.h>
#include <stdio.h>
#include <pthread.h>

#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/socket.h>
```

```c
#include <ti/display/Display.h>

#define UDPPACKETSIZE 1472
#define LISTEN_PORT   1000

extern Display_Handle display;
extern void fdOpenSession();
extern void fdCloseSession();
extern void *TaskSelf();

void *udpReceiverFxn(void *arg0)
{
    int sock = -1;
    int bytesRcvd;
    struct sockaddr_in serverAddr;
    struct sockaddr_in clientAddr;
    socklen_t addrlen;
    char buffer[UDPPACKETSIZE];

    fdOpenSession(TaskSelf());

    Display_printf(display, 0, 0, "UDP Receiver started on port %d\n", LISTEN_POR

    sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
        Display_printf(display, 0, 0, "Error: Failed to create socket.\n");
        goto shutdown;
    }

    memset(&serverAddr, 0, sizeof(serverAddr));
    serverAddr.sin_family      = AF_INET;
    serverAddr.sin_port        = htons(LISTEN_PORT);
    serverAddr.sin_addr.s_addr = htonl(INADDR_ANY);

    if (bind(sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr)) < 0) {
        Display_printf(display, 0, 0, "Error: Bind failed.\n");
        goto shutdown;
    }

    while (1) {
        addrlen = sizeof(clientAddr);
        bytesRcvd = recvfrom(sock, buffer, sizeof(buffer) - 1, 0,
                             (struct sockaddr *)&clientAddr, &addrlen);

        if (bytesRcvd > 0) {
```

```
            buffer[bytesRcvd] = '\0';
            Display_printf(display, 0, 0, "[From %s]: %s\n",
                           inet_ntoa(clientAddr.sin_addr), buffer);
        }
    }

shutdown:
    if (sock >= 0) {
        close(sock);
    }
    fdCloseSession(TaskSelf());
    return (NULL);
}
```

`udpEchoHooks.c` for Board 2

Update the entry hook call to pass execution to `udpReceiverFxn`:

C

```c
extern void *udpReceiverFxn(void *arg0);

/* Inside netIPAddrHook() ... */
retc = pthread_create(&thread, &attrs, udpReceiverFxn, NULL);
if (retc != 0) {
    Display_printf(display, 0, 0, "pthread_create failed\n");
    while (1);
}
```

i want for board to board with cable connected

When connecting two **MSP432E401Y LaunchPads directly board-to-board with an Ethernet cable** (without a router, switch, or DHCP server), here is everything you need to know and the exact code/configuration.

Key Hardware Note: The Cable

You can use a **standard, straight-through RJ45 Ethernet cable** (a crossover cable is **not** required).

The internal Ethernet PHY on the MSP432E401Y chip supports **Auto-MDIX (Automatic Medium-Dependent Interface Crossover)**, meaning the PHY automatically detects and configures the transmit and receive pairs when connected directly to another board.

Step 1: Exact SysConfig (`.syscfg`) Settings for Direct Link

Since there is **no DHCP server** on a direct cable link:

1. Open your project's `.syscfg` file.
2. Go to **TI NDK** → **General**.
3. Under **Network IP Configuration**:

   Change **IP Addressing Mode** to `Static IP` .

   Crucial: Disable the DHCP Client service (`DHCP Client` → Uncheck / Remove) so the NDK stack doesn't waste time broadcasting DHCP Discover packets or faulting when none responds.

Use the following IP parameters:

Parameter	Board 1 (Transmitter / Initiator)	Board 2 (Receiver / Responder)
IP Address	`192.168.1.101`	`192.168.1.102`
Subnet Mask	`255.255.255.0`	`255.255.255.0`
Default Gateway	`192.168.1.1`	`192.168.1.1`

(If editing `.syscfg` in text editor mode, find `Global.ndkThreadPriority` / `Global.IPv6` section and update the `ndk.General` IP address strings directly).

Step 2: Board 1 Project (Transmitter / Client)

`udpSender.c` (Replaces `udpEcho.c` in Board 1's project)

Board 1 sets up an Ethernet UDP socket, targets Board 2 at `192.168.1.102:1000`, and continuously sends packets across the direct cable link:

c

```c
#include <string.h>
#include <stdint.h>
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

/* BSD Socket headers */
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#include <ti/display/Display.h>

#define TARGET_IP     "192.168.1.102"
#define TARGET_PORT   1000
#define PACKET_BUF    128

extern Display_Handle display;
extern void fdOpenSession();
extern void fdCloseSession();
extern void *TaskSelf();

void *udpSenderFxn(void *arg0)
{
    int sock = -1;
    struct sockaddr_in destAddr;
    char buffer[PACKET_BUF];
    int count = 0;
    int bytesSent;

    /* Register task with TI-NDK stack */
    fdOpenSession(TaskSelf());

    Display_printf(display, 0, 0, "=== Direct Cable: Board 1 (Sender) Ready ===")

    sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
        Display_printf(display, 0, 0, "Error: Could not open socket\n");
```

```
            goto shutdown;
        }

        memset(&destAddr, 0, sizeof(destAddr));
        destAddr.sin_family      = AF_INET;
        destAddr.sin_port        = htons(TARGET_PORT);
        destAddr.sin_addr.s_addr = inet_addr(TARGET_IP);

        while (1) {
            snprintf(buffer, sizeof(buffer), "Ping from Board 1 [Seq: %d]", count++);

            bytesSent = sendto(sock, buffer, strlen(buffer), 0,
                               (struct sockaddr *)&destAddr, sizeof(destAddr));

            if (bytesSent < 0) {
                Display_printf(display, 0, 0, "Send failed (Cable disconnected?)\n");
            } else {
                Display_printf(display, 0, 0, "Sent (%d bytes): %s\n", bytesSent, buf
            }

            /* 1 second interval */
            sleep(1);
        }

    shutdown:
        if (sock >= 0) {
            close(sock);
        }
        fdCloseSession(TaskSelf());
        return NULL;
    }
```

In `udpEchoHooks.c` on Board 1:

Ensure the thread starts `udpSenderFxn`:

C

```
extern void *udpSenderFxn(void *arg0);

/* Inside netIPAddrHook(): replace the echoFxn thread start with this */
```

```
    retc = pthread_create(&thread, &attrs, udpSenderFxn, NULL);
    if (retc != 0) {
        Display_printf(display, 0, 0, "pthread_create failed\n");
        while (1);
    }
```

Step 3: Board 2 Project (Receiver / Listener)

`udpReceiver.c` (Replaces `udpEcho.c` in Board 2's project)

Board 2 binds to port `1000` on its static IP (`192.168.1.102`), waits on the cable interface, and prints every message received from Board 1:

C

```c
#include <string.h>
#include <stdint.h>
#include <stdio.h>
#include <pthread.h>

#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#include <ti/display/Display.h>

#define UDPPACKETSIZE 1472
#define LISTEN_PORT   1000

extern Display_Handle display;
extern void fdOpenSession();
extern void fdCloseSession();
extern void *TaskSelf();

void *udpReceiverFxn(void *arg0)
{
    int sock = -1;
    int bytesRcvd;
    struct sockaddr_in serverAddr;
    struct sockaddr_in clientAddr;
    socklen_t addrlen;
    char buffer[UDPPACKETSIZE];
```

```
    /* Register task with TI-NDK stack */
    fdOpenSession(TaskSelf());

    Display_printf(display, 0, 0, "=== Direct Cable: Board 2 (Receiver) Listening

    sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
        Display_printf(display, 0, 0, "Error: Could not open socket\n");
        goto shutdown;
    }

    memset(&serverAddr, 0, sizeof(serverAddr));
    serverAddr.sin_family      = AF_INET;
    serverAddr.sin_port        = htons(LISTEN_PORT);
    serverAddr.sin_addr.s_addr = htonl(INADDR_ANY);

    if (bind(sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr)) < 0) {
        Display_printf(display, 0, 0, "Error: Bind failed\n");
        goto shutdown;
    }

    while (1) {
        addrlen = sizeof(clientAddr);
        bytesRcvd = recvfrom(sock, buffer, sizeof(buffer) - 1, 0,
                             (struct sockaddr *)&clientAddr, &addrlen);

        if (bytesRcvd > 0) {
            buffer[bytesRcvd] = '\0';
            Display_printf(display, 0, 0, "[Received from %s]: %s\n",
                           inet_ntoa(clientAddr.sin_addr), buffer);
        }
    }

shutdown:
    if (sock >= 0) {
        close(sock);
    }
    fdCloseSession(TaskSelf());
    return NULL;
}
```

In `udpEchoHooks.c` on Board 2:

Ensure the thread starts `udpReceiverFxn`:

```c
extern void *udpReceiverFxn(void *arg0);

/* Inside netIPAddrHook(): replace the echoFxn thread start with this */
retc = pthread_create(&thread, &attrs, udpReceiverFxn, NULL);
if (retc != 0) {
    Display_printf(display, 0, 0, "pthread_create failed\n");
    while (1);
}
```

Step 4: Verification and Running

1. **Physical Link:** Plug the standard RJ45 cable directly into the Ethernet jacks of both MSP432E401Y boards.
2. **Power & LEDs:** Power both boards via USB. Look at the RJ45 jack LEDs:

   The **Green Link LED** should turn solid ON (indicating the physical link layer and Auto-MDIX established connection).

   The **Yellow/Amber Activity LED** will blink when packets are transmitted.
3. **Serial Terminal (UART):** Open two terminal windows (115200 baud, 8-N-1):

   Board 1 terminal: Will print
   `Sent (32 bytes): Ping from Board 1 [Seq: 0], Seq: 1 ...`

   Board 2 terminal: Will print
   `[Received from 192.168.1.101]: Ping from Board 1 [Seq: 0] ...`

error #10234-D: unresolved symbols remain
error #10010: errors encountered during linking;
"Reciever_udp.out" not built

The linker error `error #10234-D: unresolved symbols remain` means the compiler successfully compiled your `.c` files, but the TI ARM linker could not find the definitions (implementations) for one or more functions or variables referenced in your project.

Here are the most common causes and how to fix them for the MSP432E401Y NDK/FreeRTOS project:

Step 1: Check the Exact Missing Symbol Names

Scroll up in the **Console** tab of Code Composer Studio (CCS) directly above `error #10234-D`. Look for lines formatted like:

Plaintext

```
undefined            first referenced
  symbol                 in file
---------            ----------------
<symbol_name>        ./<file_name>.obj
```

Step 2: Fix Common Missing Symbols

1. Missing NDK Function Reference (e.g., `echoFxn` vs `udpReceiverFxn`)

Problem: In `udpEchoHooks.c`, the template originally declares and invokes `echoFxn`:

udpEchoHooks.c

C

```c
extern void *echoFxn(void *arg0);
```

If you changed the function name in `udpReceiver.c` to `udpReceiverFxn` but did not update the `extern` declaration and the `pthread_create()` call inside `udpEchoHooks.c`, the linker will complain that `echoFxn` is unresolved. udpEchoHooks.c

Fix: In `udpEchoHooks.c`:

1. Replace `extern void *echoFxn(void *arg0);` with:

```c
extern void *udpReceiverFxn(void *arg0);
```

2. In `netIPAddrHook()`, ensure the thread creation uses the updated function pointer:
udpEchoHooks.c

```c
retc = pthread_create(&thread, &attrs, udpReceiverFxn, (void *)&arg0);
```

2. Missing `udpReceiver.c` from Compilation

Problem: If you created a new source file or renamed `udpEcho.c`, it may have been excluded from the build.

Fix:

1. In the CCS **Project Explorer**, expand your project.
2. Locate `udpReceiver.c` (or your receiver source file).
3. If there is a small diagonal cross/slash through the file icon, right-click the file → **Resource Configurations** → **Exclude from Build...** → uncheck all configurations → click **OK**.

3. Missing POSIX / pthread or NDK Symbols

If the missing symbols are POSIX threads (`pthread_create`, `pthread_attr_init`), socket functions (`socket`, `bind`, `recvfrom`), or NDK functions (`fdOpenSession`, `fdCloseSession`):

Posix / FreeRTOS headers: Verify your project properties have the POSIX layer included:

Right-click project → **Properties** → **Build** → **ARM Compiler** → **Include Options**.

Ensure `${COM_TI_SIMPLELINK_MSP432E4_SDK_INSTALL_DIR}/source/ti/posix/freertos` is present in the search path.

NDK Libraries: In CCS, ensure the project's SysConfig file (`.syscfg`) is building properly, as SysConfig generates the linker command file (`ti_ndk_config.c` / `ti_ndk_config.h`) containing library link references.

Step 3: Clean and Rebuild

1. Right-click your project → **Clean Project**.
2. Right-click your project → **Rebuild Project**.

```
*** Build of configuration Debug for project Reciever_udp
****

"C:\\ti\\ccs1010\\ccs\\utils\\bin\\gmake" -k -j 12 all -C
```

The single missing symbol causing the build failure is:

```
 undefined      first referenced
  symbol            in file
---------      ----------------
 inet_ntoa     ./udp_recieve.obj
```

In the TI-NDK stack, standard BSD networking functions like `inet_ntoa` are mapped to NDK-prefixed functions (e.g., `NDK_inet_ntoa`) using macros, or handled via NDK's internal utilities. When `inet_ntoa` is called directly without the proper header mapping, the linker cannot find an un-prefixed symbol named `inet_ntoa` .

Here are two quick ways to solve this:

Solution 1: Use TI's `NDK_inet_ntoa` directly (Recommended)

1. Open `udp_recieve.c` .
2. Add the NDK header:

```c
#include <ti/ndk/inc/netmain.h>
```

3. Replace:

```c
inet_ntoa(clientAddr.sin_addr)
```

with:

```c
NDK_inet_ntoa(clientAddr.sin_addr)
```

Solution 2: Print the IP address without any helper functions

If you don't want to include extra stack headers or depend on `inet_ntoa` , unpack the 32-bit IP address directly inside your `Display_printf` call:

Replace:

```c
Display_printf(display, 0, 0, "[Received from %s]: %s\n",
               inet_ntoa(clientAddr.sin_addr), buffer);
```

With:

C

```c
uint32_t ip = ntohl(clientAddr.sin_addr.s_addr);
Display_printf(display, 0, 0, "[Received from %d.%d.%d.%d]: %s\n",
               (uint8_t)(ip >> 24) & 0xFF,
               (uint8_t)(ip >> 16) & 0xFF,
               (uint8_t)(ip >> 8)  & 0xFF,
               (uint8_t)(ip)       & 0xFF,
               buffer);
```

Minor Cleanup (Optional)

The compiler also reported this warning:

Plaintext

```
"../udpEchoHooks.c", line 40: warning #179-D: variable "arg0" was declared
but never referenced
```

In `udpEchoHooks.c`, you can either delete `static uint16_t arg0 = UDPPORT;` or pass it into `pthread_create` as `(void *)&arg0` to silence the warning.

Description Resource Path Location Type
<a
href="file:/C:/ti/ccs1010/ccs/tools/compiler/dmed/HTML/225.html">#225-D function "inet_addr" declared implicitly
udpsender.c /Transmitter_udp line 45 C/C++ Problem

The warning `#225-D: function "inet_addr" declared implicitly` occurs because the TI-NDK stack headers declare BSD conversion functions with the `NDK_` prefix (e.g., `NDK_inet_addr`) unless the NDK's BSD compatibility mapping header is explicitly included, or the prefixed name is used directly.

Here are two ways to fix this in `udpsender.c`:

Method 1: Use `NDK_inet_addr` with NDK Headers (Recommended)

1. Open `udpsender.c`.
2. Add the TI-NDK main header near the top of your include list:

```c
#include <ti/ndk/inc/netmain.h>
```

3. Update line 45 to use `NDK_inet_addr`:

```c
destAddr.sin_addr.s_addr = NDK_inet_addr(TARGET_IP);
```

Verification: Build the project (`Ctrl + B`). The `#225-D` warning for `inet_addr` will disappear.

Method 2: Convert the Static IP Directly Without Calling Functions

If you prefer to avoid calling conversion functions altogether, parse the fixed IP string directly into its 32-bit big-endian representation:

Replace line 45:

```c
```

```
destAddr.sin_addr.s_addr = inet_addr(TARGET_IP);
```

With:

```c
/* 192.168.1.102 in network byte order */
destAddr.sin_addr.s_addr = htonl((192U << 24) | (168U << 16) | (1U << 8) | 102U);
```

Verification: Build the project. It removes the dependency on `inet_addr` and compiles cleanly without implicit declaration warnings.

Description Resource Path Location Type
#179-D variable "arg0" was declared but never referenced
udpEchoHooks.c /Transmitter_udp line 43 C/C++ Problem

This compiler warning occurs because `arg0` was declared as a static variable inside `udpEchoHooks.c` but is never passed or accessed anywhere in the function.

In the original SDK template, `arg0` holds the port number and is passed as the thread argument to the worker task:

```c
static uint16_t arg0 = UDPPORT;
```

When you switched the task to `udpSenderFxn` and passed `NULL` as the parameter in `pthread_create()`, `arg0` was left unused.

Option 1: Delete the unused variable (Cleanest)

Open `udpEchoHooks.c` on the transmitter project:

1. Locate line 43:

```c
static uint16_t    arg0 = UDPPORT;
```

2. Delete that line entirely.
3. Rebuild the project (`Ctrl + B`).

Verification: The `#179-D` warning will be gone from the Problems tab.

Option 2: Pass `arg0` into `pthread_create`

If you prefer keeping the variable for future port configuration, pass its address into `pthread_create()` instead of `NULL` :

Find the `pthread_create` call inside `netIPAddrHook()` :

```c
retc = pthread_create(&thread, &attrs, udpSenderFxn, (void *)&arg0);
```

Verification: Rebuild the project. The compiler sees `arg0` being referenced and clears the warning.

If you encounter any other build warnings or runtime issues:

Packets not arriving ↪	Check link status ↪